ASSIGNMENT-13.2

Roll no:2303A52074
Batch-37

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

PROMPT

Refactor this Python code by removing repeated logic.
Create a reusable function to calculate area and perimeter of a rectangle.
Ensure the output remains the same and add proper docstrings.

CODE

ai 13.2.py > ...

```python
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
#Refactor this Python code by removing repeated logic.
#Create a reusable function to calculate area and perimeter of a rectangle.
#Ensure the output remains the same and add proper docstrings.
def calculate_area_and_perimeter(length, width):
    """
    Calculates the area and perimeter of a rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
area, perimeter = calculate_area_and_perimeter(length, width)
print("Area:", area)
print("Perimeter:", perimeter)
```

Ctrl+L to chat, Ctrl+K to generate

OUTPUT

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
Area: 72
Perimeter: 36
```

EXPLANATION

The original program repeated the same calculations for area and perimeter multiple times, which makes the code longer and harder to maintain.

To improve the code, the repeated logic was placed inside a reusable function so it can be called whenever needed.

This refactoring makes the program cleaner, easier to read, and ensures the output remains the same

Task Description -2 (Refactoring – Modularizing Inline Calculations)

PROMPT

Analyze the following Python code and identify repeated inline calculations.

Refactor the program by converting the repeated discount calculation into a reusable function.

Ensure the code is modular, readable, and includes proper docstrings explaining the function.

CODE

```python
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
#Analyze the following Python code and identify repeated inline calculations.
#Refactor the program by converting the repeated discount calculation into a reusable function.
#Ensure the code is modular, readable, and includes proper docstrings explaining the function.
def calculate_discount(amount):
    return amount * 0.10
    final_amount = amount - discount
    return final_amount
final_amount = calculate_discount(1000)
print("Final Amount:", final_amount)
final_amount = calculate_discount(2000)
print("Final Amount:", final_amount)
#add docstrings to the function
def calculate_discount(amount):
    """
    Calculates the discount of an amount.
    """
    return amount * 0.10
    final_amount = amount - discount
    return final_amount
final_amount = calculate_discount(1000)
print("Final Amount:", final_amount)
final_amount = calculate_discount(2000)
print("Final Amount:", final_amount)
```

OUTPUT

```
Final Amount: 900.0
Final Amount: 1800.0
Final Amount: 100.0
Final Amount: 200.0
Final Amount: 100.0
Final Amount: 200.0
```

EXPLANATION

First, I asked the AI to analyze the code and refactor the repeated discount calculation into a reusable function.The AI generated the function but did not include a docstring initially.So I asked the AI again to add a docstring, and it updated the function with proper documentation.

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

PROMPT

Analyze the following code that uses nested loops and conditional checks to find names in a list. Suggest a more efficient approach to improve performance using better data structures or simplified logic. Refactor the code while preserving the same output and include a small comparison of execution efficiency before and after optimization.

CODE

```python
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
#Analyze the following code that uses nested loops and conditional checks to find names in a list.
#Suggest a more efficient approach to improve performance using better data structures or simplified logic.
#Refactor the code while preserving the same output and include a small comparison of execution efficiency b
def find_names(students, check_list):
    for name in check_list:
        if name in students:
            print(name, "found")
        else:
            print(name, "not found")
find_names(students, check_list)
#compare the execution efficiency before and after optimization
import time
start_time = time.time()
find_names(students, check_list)
end_time = time.time()
print("Execution time:", end_time - start_time)
start_time = time.time()
find_names(students, check_list)
end_time = time.time()
print("Execution time:", end_time - start_time)
```

OUTPUT

```
Charan found
Esha not found
Bhavya found
Charan found
Esha not found
Bhavya found
Charan found
Esha not found
Bhavya found
Execution time: 0.0005640983581542969
Charan found
Esha not found
Bhavya found
Execution time: 0.0005893707275390625
```

EXPLANATION

First, I asked the AI to analyze the code and identify inefficient parts like nested loops and repeated condition checks.The AI suggested using a better data structure and simplified logic in operator to make the search faster.The refactored code removes nested loops, improves performance, and keeps the same output while running more efficiently.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

PROMPT

Refactor the following Python code by replacing all hardcoded numeric values used in the simple interest calculation with descriptive constants . Define these constants at the beginning of the program and update the calculations to use them. Return the complete refactored Python program (not partial edits) and keep the program output the same.

CODE

```python
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
#Refactor the following Python code by replacing all hardcoded numeric values used in the simple interest
#Define these constants at the beginning of the program and update the calculations to use them.
#Return the complete refactored Python program (not partial edits) and keep the program output the same.
RATE = 0.05
TIME = 2
print("Simple Interest:", (1000 * RATE * TIME) / 100)
print("Simple Interest:", (2000 * RATE * TIME) / 100)
#compare the execution efficiency before and after optimization
import time
start_time = time.time()
print("Simple Interest:", (1000 * RATE * TIME) / 100)
end_time = time.time()
print("Execution time:", end_time - start_time)
start_time = time.time()
print("Simple Interest:", (2000 * RATE * TIME) / 100)
end_time = time.time()
print("Execution time:", end_time - start_time)
```

OUTPUT

```
Simple Interest: 100.0
Simple Interest: 200.0
Simple Interest: 1.0
Simple Interest: 2.0
Simple Interest: 1.0
Execution time: 0.00017452239990234375
Simple Interest: 2.0
Execution time: 0.0002620220184326172
```

EXPLANATION

First, I gave the original code with hardcoded numbers to the AI and asked it to refactor it by replacing those values with constants.
The AI defined RATE and TIME at the beginning of the program and updated the simple interest calculations to use these constants instead of direct numbers.
After that, I added execution time measurement using the TIME module to check the performance of the calculation after refactoring.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

PROMPT

#Analyze the following Python code and improve its readability by replacing unclear variable names with meaningful identifiers.
#Add appropriate inline comments to explain the logic while keeping the program output the same. #Return the complete refactored Python program.
#Return the complete refactored Python program.

CODE

```
x = 50
y = 30
z = x + y
print(z)
#Analyze the following Python code and improve its readability by replacing unclear variable names with mea
#Add appropriate inline comments to explain the logic while keeping the program output the same.
#Return the complete refactored Python program.
num1 = 50
num2 = 30
sum_result = num1 + num2
print(sum_result)
#compare the execution efficiency before and after optimization
import time
start_time = time.time()
print(sum_result)
end_time = time.time()
print("Execution time:", end_time - start_time)
start_time = time.time()
print(sum_result)
end_time = time.time()
print("Execution time:", end_time - start_time)
'''
```

OUTPUT

```
80
80
80
Execution time: 0.00025200843811035156
80
Execution time: 0.00014352798461914062
```

EXPLANATION

First, I asked the AI to analyze the code and replace unclear variable names like x, y, and Z.The AI changed them to meaningful names such as num1, num2, and sum_result.It also added comments to explain the steps, making the code easier to read while keeping the same output.